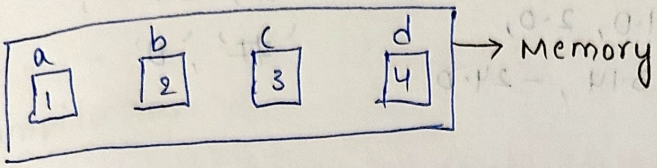
C - LANGUAGE

CH-1 : VARIABLES, DATATYPES, INPUT/OUTPUT

variables :- Variable is the name of a memory location which stores some data.



1, 2, 3, 4 → data
a, b, c, d → variable

```
# include < stdio.h >
int main() {
int number = 25;
char star = '*';
float pi = 3.14;
return 0;
}
```

int → Data type
number → variable
25 → Data

Rules :-

a). variables are case sensitive. (a and A are different)
b). 1st character is alphabet or '_'.
c). no commal blank place
d). no other symbol other than '_'.

Data type	Size in byte
char	1
float	4
int	2
double	8

Constants:- Values dont change. (fixed)

Types

Integer constants	Real constants	Character constants
1, 2, 3, 0, -1, -2	1.0, 2.0, 3.14, -24.0	'a', 'b', '*', '#', 'y'

Program structure:-

```
# include<stdio.h>   → preprocessor directive
int main() {
Printf ("Hello World");
return 0; }
```

Comments:- Lines that are not part of program

→ Single line //

→ Multiple line /* */

Output:-

```
Printf ("Hello World\n");   (new line)
```

Cases:-

1). **Integers:-**

```
Printf ("age is %d", age);
```

2). **Real nos:-**

```
printf (" Value of pi is %.f", pi);
```

3. characters :-
printf(" star looks like this %c", 'star');

Input :-
scanf("%d", &age);
↳ address

Compilation :-
A computer program that translates 'C' code into machine code

Hello.c ⟶ C compiler ⟶ a.exe (windows)
a.out (linux & mac)

P-Q-1 :- write a program to calculate area of square

```
#include <stdio.h>
int main() {
int side;
Printf("Enter side");
scanf("%d", &side);
printf("The area of Square is %d", side * side);
return 0; }
```

P-Q-2 :- write a program to calculate area of circle.

```
#include<stdio.h>
int main() {
float rad;
Printf("Enter radius");
scanf("%f", &rad);
Printf("Area of circle is %f ", 3.14 * rad * rad);
return 0;
}
```

P-Q-3:- Write a program to calculate perimeter of rectangle.

```
#include <stdio.h>
int main() {
int a, b;
Printf("Enter a");
Scanf("%d", &a);
Printf("Enter b");
Scanf("%d", &b);
Printf("Perimeter of rectangle is %d", 2*(a+b));
return 0; }
```

P-Q-4 :- Take a number from the user and output its cube.

```
#include <stdio.h>
int main() {
int n;
Printf("Enter n");
Scanf("%d", &n);
Printf("cube is %d", n*n*n);
return 0;
}
```

CH-2 : INSTRUCTIONS & OPERATORS

Instructions :-

These are statements in a program.

Types

- Type declaration instructions
- Arithmetic instructions
- Control instructions

Type declaration :-

Declare variabe before using it

valid

```
int a = 22;
int b = a;
int c = b+1;
int d = 1, e;

int a,b,c;
a=b=c=1;
```

invalid

```
int a = 22;
int b = a;
int c = b+2;
int d = 2,e;

int a,b,c=1;
```

Arithmetic Operations :-

a + b

operand1 operator operand2

valid

```
a=b+c
a=b*c
a=b/c
```

invalid

```
b+c=a
a=bc
a=b^c
```

⇒ Pow(x,y) = x^y

Modular operator :- Returns remainder for int

3 % 2 = 1

Type conversion:-

int op int → int

int op float → float

float op float → float

conversion → implicit (compiler do)

conversion → explicit (programmer do)

Ps-5 :- Solve : int a = 1.999999;

```
#include <stdio.h>
int main() {
int a = (int)1.999999;
Printf("%d\n", a);
return 0; }
```

---

operator precedence :-

* / %

↓

\+ −

↓

=

x = 4 + 9 * 10

= 4 + 90

= 94

x = 4 * 3 / 6 * 2

= 12 / 6 * 2

= 2 * 2 = 4

(If same precedence then left to right)

Control instructions :-

Used to determine flow of program

a). Sequence control
b). Decision control
c). loop control
d). case control

Operators :-

a). Arithmetic Operators (+, -, /, *, %)
b). Relational operators
c). Logical operators
d). Bitwise operators
e). Assignment operators
f). Ternary operator

---

b). Relational operators :-

==
>, >=
<, <=
!=

```
#include <stdio.h>
int main() {
Printf ("%d\n", 4==4);
return 0; }
```

output :- 1 (i.e true)

```
Printf ("%d\n", 4==3);
Printf ("%d\n", 4>=3);
Printf ("%d\n", 3>4);
Printf ("%d\n", 4!=4);
```

output :- 0
output :- 1
output :- 0
output :- 0

(c) · logical operators :-

&& AND

|| OR

! NOT

```
#include<stdio.h>
int main() {
Printf("%d\n", 3>4 && 5>2);
return 0; }
```

0 1

&&

1	2	out
T	T	T
T	F	F
F	T	F
F	F	F

||

1	2	out
T	T	T
T	F	T
F	T	T
F	F	F

```
Printf("%d\n", !(5>1));   ⇒ output = 0
Printf("%d\n", !((5>1)&&(3>4)));
                          0 ⇒ !(0) = 1
```

Operator precedence :-

Priority	Operator
1	!
2	*, /, %
3	+, -
4	<, <=, >=, >
5	==, !=
6	&&
7	||
8	=

(e). Assignment operators :-

= → assigning the values from right hand to left hand.

+= (a = a+b)

-= (a = a-b)

*= (a = a*b)

/= (a = a/b)

%= (a = a%b)

P-Q-6 :- write a program to check if a number is divisible by 2 or not.

```
#include<stdio.h>
int main() {
int x;
printf("Enter a number: ");
scanf("%d", &x);
printf("%d", x % 2 == 0);
return 0; }
```

P-Q-7 :- Are the following valid or not

a). int a = 8 ^ 8 (XOR) ⇒ output = 0 (valid)

b). int x; int y = x; valid

c). int x@y = x; invalid

d). char stars = '**' invalid

P-Q-8 :- print 1 (true) or 0 (false) for following statements:

a). If it's sunday & it's snowing ⇒ true

b). If it's monday or its raining ⇒ true.

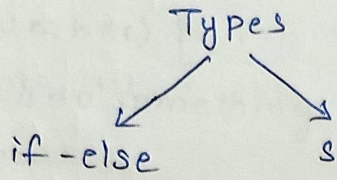
c). If a number is > 9 & < 100 ⇒ true.

a).
```
# include<stdio.h>
int main (){
int isSnowing = 1;
int isSunday = 1;
Printf ("%d\n", isSunday && isSnowing);
return 0; }
```

b).
```
# include<stdio.h>
int main (){
int isMonday = 1;
int isRaining = 1;
Printf("%d\n", isMonday || isRaining);
return 0; }
```

c).
```
# include<stdio.h>
int main (){
int x;
Printf("Enter number: ");
scanf ("%d", &x);
Printf("%d\n", x>9 && x<100);
return 0; }
```

CH-3 : CONDITIONAL STATEMENTS



---

if-else:-

```
if (condition){
  // do something if True
}
else {
  // do something if False
}
```

```
#include <stdio.h>
int main() {
 int age;
 Printf("Enter age: ");
 Scanf("%d", &age);
 if (age > 18) {
   Printf("adult\n");
  }
 else {
  Printf("not adult\n");
  }
 return 0;
 }
```

---

else if:-

```
if (condition 1) {
  // do something if true
 }
else if (condition 2) {
 // do something if 1st is false & 2nd is true
}
```

```c
#include <stdio.h>
int main() {
int age;
printf("Enter age: ");
scanf("%d", &age);
if(age >= 18) {
 printf("adult \n");
 }
else if (age >13 && age < 18) {
 printf("teenager \n");
 }
 else {
   printf("child");
  }
 return 0;
 }
```

Ternary:-

condition ? dosomething if true : dosomething if false

```c
# include <stdio.h>
 int main() {
 int age;
 printf("Enter age: ");
 scanf("%d", &age);
 age >= 18 ? printf("adult\n"): printf("not adult\n");
 return 0;
 }
```

Switch :-

```
Switch (number) {
case (1 : // do something
          break ;
case (2 : // do something
          break;
default : //do something
 }
```

```
# include < stdio.h>
int main () {
int day ;
printf(" Enter day (1-7): ");
scanf("%d", & day);
switch (day) {
  case 1 : printf(" monday");
           break;
  case 2 : printf(" Tuesday");
           break;
  case 3 : printf ("wednesday");
           break;
  case 4 : printf (" Thursday");
           break;
  case 5 : printf (" Friday");
           break;
  case 6 : printf (" Saturday");
           break;
  case 7 : printf (" Sunday");
           break;
default : printf(" not a valid day ");
}
```

Switch properties :-

(a). cases can be in any order

(b). Nested switch (switch inside switch) are allowed.

```
#include <stdio.h>
 int main ( ) {
  int number;
 Printf ("Enter a number : ");
 Scanf ("%d", & number);
 if (number > = 0) {
 Printf ("positive\n");}
   if (number % 2 = =0) {
       Printf("even\n");
     }
   else{
     Printf ("odd\n");
     }

else {
  Printf("negative\n");
   }
 return0;
 }
```

P-Q-9 :- write a program to check if a student passed or failed marks <=30 ⇒ fail
marks > 30 ⇒ pass

```
#include< stdio.h>
 int main() {
 int marks;
 Printf ("Enter marks (0-100):  ");
 Scanf ("%d", & marks);
 if(marks <=30) {
    Printf("FAIL\n");
     }
```

```
else if (marks > 30 && marks <= 100){
    printf (" PASS \n ");
}
else {
 printf (" wrong marks");
  }
 return 0;
  }
```

(or)

```
marks <= 30? printf("FAIL \n") : printf(" PASS\n");
```

P-Q-10:- write a program to give grades to a student

marks < 30 is C
30 <= marks < 70 is B
70 <= marks < 90 is A
90 <= marks <= 100 is A+

```
#include <stdio.h>
int main() {
  int marks;
  Printf("enter number (0-100):");
  scanf ("%d", &marks);
  if (marks < 30) { Printf("C\n"); }
  else if (marks >= 30 && marks < 70) {
   Printf("B\n"); }
  else if (marks >= 70 && marks < 90){
   Printf( "A+\n"); }
  return 0;
  }
```

P-Q-11 :-

Will this code:

```
int x = 2;
if (x = 1) {
Printf (" x is equal to 1 ");
} else {
Printf (" x is not equal to 1");
}
```

(a). give error

(b). print " x is equal

(c). print " x is not eq to 1

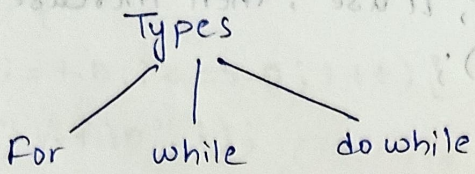
P-Q-12 :-

write a program to find the character entered by user upper case or not.

```
#include < stdio.h >
int main() {
char ch;
Printf(" enter character : ");
scanf("%c", &ch);
if(ch >='A' && ch <='Z') {
    Printf("upper case");
  }
else if(ch >='a' && ch <='z') {
    Printf("lower case");
  }
else {
    Printf (" not english letter");
  }
```

CH-4 : LOOP CONTROL STATEMENTS

To repeat some parts of the program



for loop :-

```
for (intialization ; condition ; updation) {
    // do something
}
```

```
#include<stdio.h>
int main() {
 for (int i=1 ; i<=5 ; i=i+1) {
   printf ("Hello world ");
   }
 return 0;
 }
```

```
#include<stdio.h>
int main(){
 for(int i=1; i<=1000; i=i+1){
   printf("%d\n", i);
   }
 return 0;
 }
```

P-Q-13 :- Print the numbers from 0 to 10

```
#include<stdio.h>
int main() {
 for(int i=0; i<=10 ; i=i+1) {
   printf("%d\n");
   }
 return 0;
 }
```

Increment operator (i++ , ++i)

```
int i = 1;   (Post increment)
Printf("%d", i++); // use , then increase
Printf("%d\n" i);
```

Output :- 1
2

(Pre increment)

```
int i = 1;
Printf("%d", ++i);  // increase , then use
Printf("%d\n" i);
```

Output :- 2
2

Decrement operator :- (--i , i--)

(Post decrement)

```
int i = 1;
Printf("%d\n", i--);
Printf("%d\n", i);
```

Output :- 1
0

(Pre decrement)

```
int i = 1;
Printf("%d\n", --i);
Printf("%d\n", i);
```

Output :- 0
0

(Loop counter can be float or even character)

```
#include<stdio.h>
int main() {
for(float i=1.0;i<=5.0;i++) {
  printf("%f\n",i);
 }
```

1.0
2.0
3.0
4.0
5.0

```
for(char ch='a';ch<='z';ch++) {
  printf("%c\n",ch);
  }
return 0;
}
```

a
b
c
d
:
z

Infinite loop :-

```
#include<stdio.h>
intmain(){
 for(int i=1;;i++){
 printf("hello world\n");
  }
 return 0;
 }
```

While loop :-

```
while(condition) {
  // do something
  }
```

```
#include<stdio.h>
int main() {
 int i=1;
 while(i<=5) {
  printf("%d\n");
    i++;
 }
return 0;
 }
```

P-Q-14 :- Print the numbers from 0 to n, if n is given by user

```
#include <stdio.h>
int main(){
int n;
printf("enter number : ");
scanf("%d", &n);

int i = 0;
while (i <= n) {
    printf("%d\n", i);
    i++;
}
return 0;
}
```

```
for(int i=0; i<=n; i++){
    printf("%d\n", i);
```

Do while loop :-

```
do {
  // do something
} while (condition);
```

```
#include <stdio.h>
int main() {
    int i = 1;
    do {
        printf("%d\n", i);
        i++;
    } while(i <= 5);
    return 0;
}
```

P-Q-15:- Print the sum of first n Natural Numbers.

```
#1 include < stdio.h>
int main () {
   int n ;
   Printf ("enter number : ");
   Scanf ("%d", & n);

   int sum = 0 ;
for (int i = 1 ; i <= n ; i++) {
   sum = sum + i ;
   }
Printf ("Sum is %d", sum);
return 0 ;
```

```
for (int i=n ; i >= 1 ; i--) {
   Printf("%d.\n", i);
}
```

P-Q-16:- Print the table of a number input by the user.

```
# includecstdio.h>
int main() {
int n;
Printf ("Enter number : ");
scanf ("%d ", & n);
for (int i = 1 ; i <= 10 ; i++) {
   printf ("%d\n", n * i);
   }
return 0;
}
```

P-Q-17:- Keep taking numbers from the user unit user enters an odd number.

```
# include < stdio.h>
 int main() {
 int n;
do {
 Printf ("enter number: ");
 scanf ("%d ", & n);
 Printf ("%d\n", n);
```

```
if (n % 2 != 0) {
    break;
}
while (1);
printf("thank you");
```

P-Q-18 :-

Keep taking numbers as input from user until user enters a number which is a multiple of 7.

```
#include <stdio.h>
int main() {
int n;
do {
printf("Enter number : ");
scanf("%d \n", n);
if (n % 7 == 0) {
    break;
}}
while (1);
printf("thank you");
return 0;
}
```

Continue :-

Continue statement → skip to next iteration

Ex:-
```
#include <stdio.h>
int main() {
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
printf("%d\n", i);}
return 0;
}
```

Output :-

1
2
4
5

P. Q-20 :-

Print all the odd numbers from 5 to 50.

```
#include<stdio.h>
int main(){
for(int i=5; i<=50; i++){
    if(i%2!=0){
        printf("%d\n",i);
    }
return 0;
}
```

P-Q-21 :- Print the factorial of a number n.

```
#include<stdio.h>
int main(){
 int n;
 Printf("Enter number");
 scanf("%d",&n);
 int fact = 1;
 for(int i=1; i<=n; i++){
    fact = fact * i;
  }
 printf("final factorial is %d",fact);
 return 0;
}
```

P-Q-22 :- print reverse of the table for a number n.

```
#include<stdio.h>
int main(){
 int n;
 Printf("enter number: ");
 scanf("%d", &n);
 for(int i=10; i>=1; i--){
    printf("%d\n", n*i);
  }
 return 0;}
```

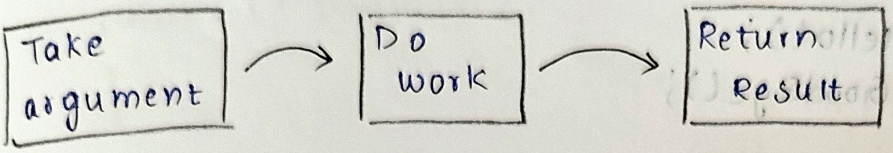
P-Q-23 :-

Calculate the sum of all numbers between 5 and 50 (including 5 and 50).

```
#include <stdio.h>
int main(){
int sum = 0;
for (int i=5; i<=50; i++){
    sum+ = i;
  }
Printf (" sum is '%d", sum);
return 0;
}
```

CH-5 : FUNCTIONS & RECURSIONS

Function :- Block of code that performs particular task.



It can be used multiple times
Increase code reusability

Syntax :-

1). **Function Prototype / declaration**

```
Void PrintHello ();
```

2). **Function Definition**

```
void printHello () {
    printf (" Hello");
}
```

3). **Function Call**

```
int main () {
    printHello();
    return 0;
}
```

```
# include < stdio.h>
void printHello(); // declaration/prototype
int main() {
    printHello (); // function call
    return 0;
}
void print Hello() { // function definition
    Printf (" Hello");
}
```

P-Q-25 :-

Write 2 functions - one to print "Hello" & second to print "good bye"..

```
#include <stdio.h>

void printHello();
void printGoodBye();

int main(){
    printHello();
    printGoodBye();
}

void printHello(){
    printf("Hello");
}

void printGoodBye(){
    Printf("GoodBye");
}
```

P-Q-26 :-

Write a function that prints Namaste if user is Indian & Bonjour if the user is french.

```
#include <stdio.h>
void namaste();
void bonjour();
int main(){
printf("enter f for french and i for indian");
char ch;
scanf("%c", &ch);
if(ch=='i'){
    printf("namaste"); namaste();
} else { printf("bonjour"); }
                bonjour();
return 0;
```

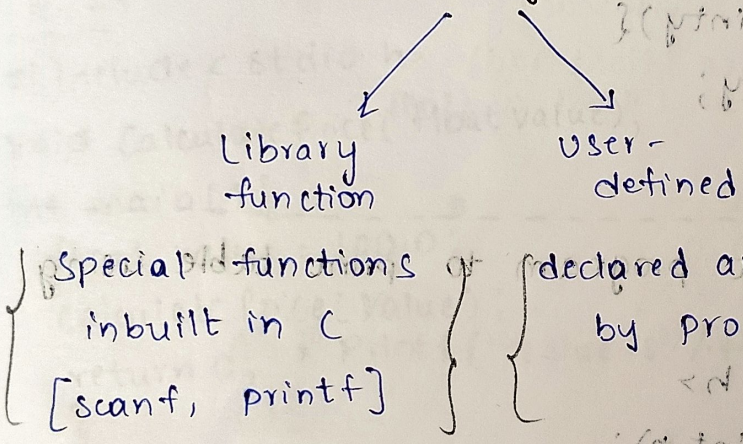
```
void namaste() {
    printf("namaste");
}
void bonjour() {
    printf("bonjour");
}
```

---

properties :-

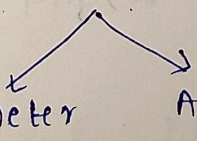
→ Execution always starts from main.
→ A function gets called directly or indirectly from main
→ There can be multiple functions in a program.

Function types



Passing Arguments :-

functions can take value and give some value.



→ void printHello();
→ void printTable(int n);
→ ~~void~~ int sum(int a, int b);

P-Q-27:-

```
#include<stdio.h>
int sum(int a, int b);
int main(){
 int a, b;
 printf("Enter a:  ");
 scanf("%d", &a);
 printf("Enter b:  ");
 scanf("%d", &b);
 int s = sum(a,b);
printf("sum is %d", s);
 return 0;
 }
int sum(int x, int y){
     return x+y;
   }
```

P-Q-28:- Write a program to print table using functions

```
#include<stdio.h>
void printTable(int n);
int main(){
 int n;
 printf("enter number: ");
  scanf("%d", &n);
  return
  printTable(n);
  return 0;}
int sum(in
void printTable(int n){
 for(int i=1; i<=10; i++){
     printf("%d", i*n);
   }
}
```

Argument V/S Parameter

Argument	Parameter
→ values that are passed in function call	→ values in function declaration & definition.
→ used to send value	→ used to receive value
→ actual parameter	→ formal parameters

Note :-

a). Function can only return one value at a time

b). Changes to parameters in function don't change the values in calling function (Because a copy of argument is passed to the argument).

P-Q-29 :-

```
#include<stdio.h>
void CalculatePrice(float value);
int main(){
  float value = 100.0;
  CalculatePrice(value);
  return 0;
}
void CalculatePrice(float value){
  value = value + (0.18 * value);
  printf("final price is : %f", value);
}
```

Output :-

final price is 118.000000

value is 100.000000

P-Q-30:- use library function to calculate the square of a number given by user.

```
#include<stdio.h>
#include<math.h>

int n=4;
printf("%f", pow(n,2));
return 0;
}
```

P-Q-31:- write functions to calculate the area of square circle and rectangle.

```
#include<stdio.h>

float squareArea(float side);
float ~~squa~~circleArea(float rad);
float rectangleArea(float a, float b);
int main(){

    float a = 5.0;
    float b = 10.0;
    float side = 2.0;
    float rad = 1.0;
squareArea(side);
circleArea(rad);
rectangleArea(a,b); }

float squareArea(float side){
    return side * side;
}

float circleArea(float rad){
    return 3.14 * rad * rad;
}

float rectangleArea(float a, float b){
    return a * b;
}
```

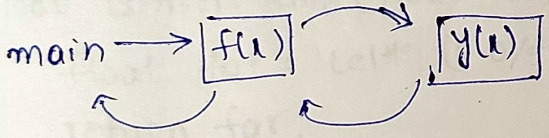
Recursions:-

When a function calls itself, it's called Recursion.

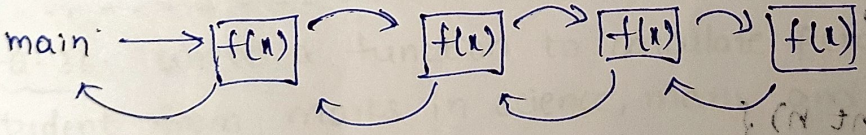
P-Q-32:- Print "Hello world" 5 times

```
#include <stdio.h>
void printHW(int count);
int main() {
    printHW(10);
    return 0;
}
void printHW (int count){
    if (count == 0){
        return;
    }
    printf("Hello World\n");
    printHW (count -1);
}
```

Normal function call



Recursion function call

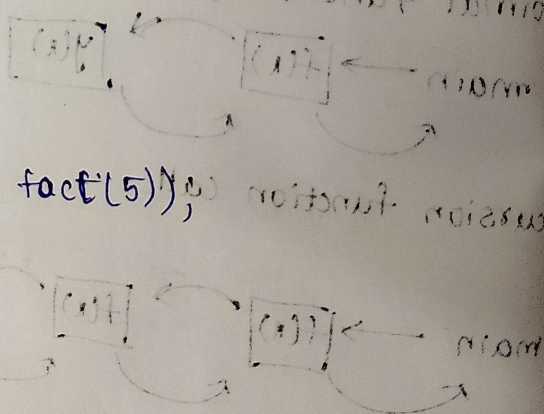


P-Q-33 :- Sum of first n natural numbers

```
#include<stdio.h>
int sum(int n);
int main() {
    Printf("Sum is %d", sum(5));
    return 0;
    }
int sum(int n) {
    if (n==1) {
    return 1;
    }
int sumNm1 = sum(n-1);
int sumN = sumNm1 + n;
return sumN;
}
```

P-Q-34 :- Factorial of n.

```
#include <stdio.h>
int fact(int n);
int main() {
    Printf("factorial is %d", fact(5));
    return 0;
}
int fact(int n) {
    if (n==0) {
      return 1;}
  int factNm1 = fact(n-1);
  int factN = factNm1 * n;
  return factN;
}
```



Properties of recursion :-

1). Anything that can be done with iteration, can be done with recursion and vice versa.

2). Recursion can sometimes give the most simple solution.

3). Base Case is the condition which stops recursion.

4). Iteration has infinite loop & Recursion has stack overflow.

P-Q-35 :- Write a function to convert celcius to fahrenheit.

```
#include <stdio.h>
float convertTemp(float cel);
int main(){
  float far = convertTemp(37);
  printf("far : %f", far);
  return 0;
}
float convertTemp(float cel){
    float far = cel* (9.0/5.0) + 32;
    return far;
}
```

P-Q-36 :- Write a function to calculate percentage of a student from marks in science, math and sanskrit.

```
#include<stdio.h>
int calcper(int sc, int math, int san);
int main(){
int float sc = 98;
    int math=95;
    int san = 99;
printf("percentage is %d", calcper(int sc, math, san));
return 0;}
int calcper(int sc, int math, int san){
    return((sc + math+san)/3) * 100;
}
```

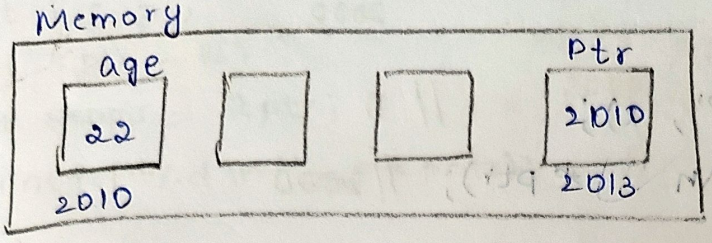
P-Q-37:- Write a function to print n terms of the fibonacci sequence.

```
#include <stdio.h>
int fib(int n);
int main(){
  fib(6);
  return 0;}

int fib(int n){
  if(n==0){
     return 0;
  }
  if(n==1){
    return 1;
  }
int fibNm1 = fib(n-1);
int fibNm2 = fib(n-2);
int fibN = fibNm1 + fibNm2;
Printf("fib of %d is %d\n", n, fibN);
return fibN;}
}
```

CH-6 : POINTERS

Pointers :- A variable that stores the memory address of another variable.

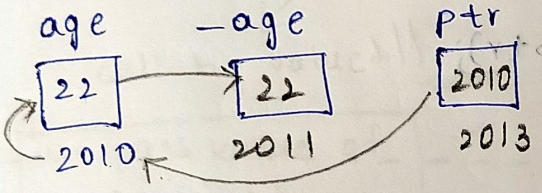


Syntax :-

```
int age = 22 ;
int *ptr = &age ;
int -age = *ptr ;
```

* ⇒ value at address

& ⇒ addresse



Declaring Pointers :-

```
int *ptr ;
char *ptr ;
float *ptr ;
```

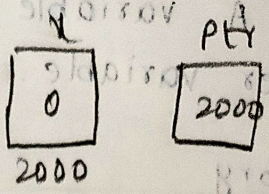
Format specifier :-

```
printf ("%p", &age);
printf ("%p", ptr);      } same output
printf ("%p", &ptr);  → pptr
```

```
Printf ("%d" , age);
printf ("%d", *ptr);
printf ("%d", *(&age));   } same output
```

P-Q-38 :-

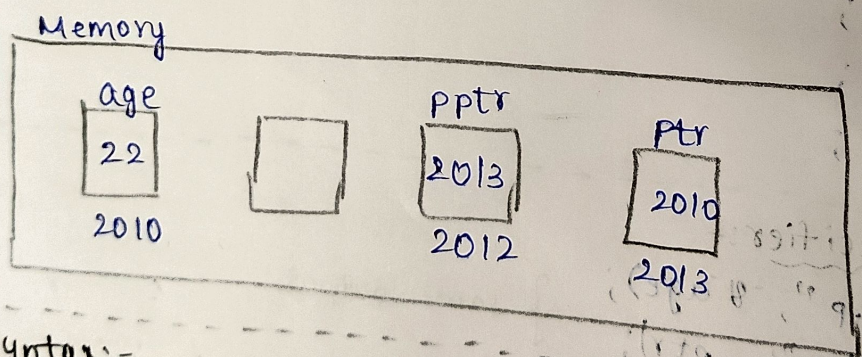
```
int *ptr;
int x;
ptr = &x;
*ptr = 0;
printf("x = %d\n", x);        // 0
printf("*ptr = %d\n", *ptr);  // 0
*ptr += 5;
printf("x = %d\n", x);        // 5
printf("*ptr = %d\n", *ptr);  // 5
(*ptr)++;
printf("x = %d\n", x);        // 6
printf("*ptr = %d\n", *ptr);  // 6
```



Find output :-

Pointer to pointer :-

A variable that stores the memory address of another Pointer

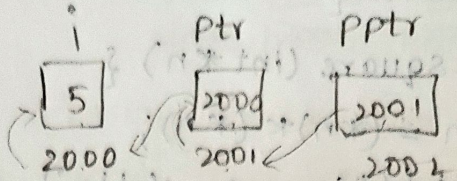


Syntax :-

```
int **pptr;
char **pptr;
float **pptr;
```

P-Q-39 :- print the value of 'i' from ~~the~~ its pointer to pointer.

```
#include <stdio.h>
int main() {
  int i = 5;
  int *ptr = &i;
  int **pptr = &ptr;
  printf ("%d\n", **pptr);   //5
  return 0;
}
```



---

pointers in function call

call by value

we pass value of variable as argument

call by reference

we pass address of variable as argument

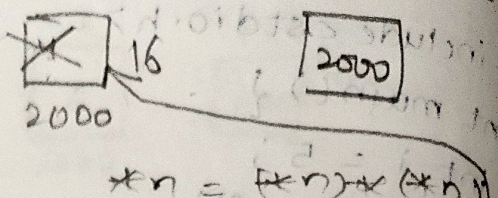
```
#include <stdio.h>
void square(int n);
int main() {
  int number = 4;
  square(number);          -> argument
  printf ("number = %d\n", number);
  return 0;
}
void square(int n) {       -> parameter
  n = n*n;
  printf ("square = %d\n", n);
}
```

parameter makes a copy of argument

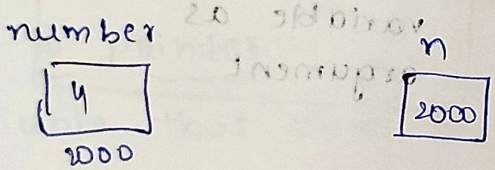
The changes made here were the reason for changes in copy

Output :- square = 16
number = 4

```
#include <stdio.h>
void square(int *n);
int main(){
void square(int *n) {
    *n = (*n)*(*n);
    printf("square = %d\n", *n);
}
int main() {
    int number = 4;
    square(&number);
    printf("number = %d\n", number);
    return 0;
}
```



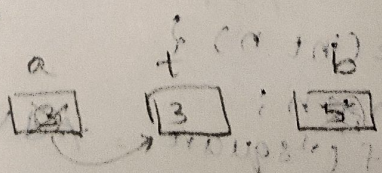
Output :- square = 16
number = 16



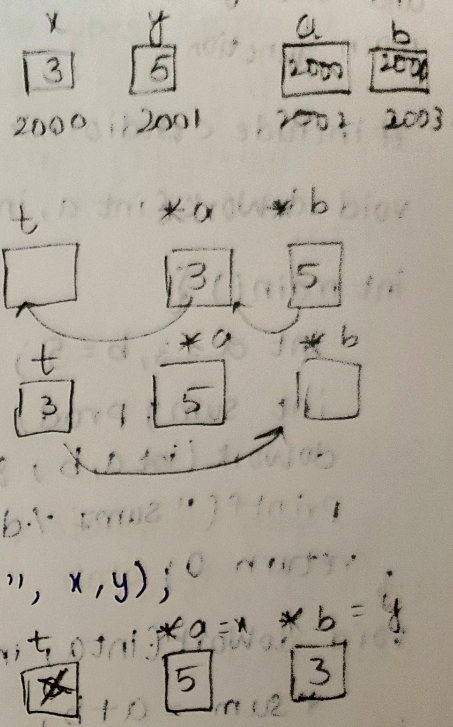
P-Q-40:- Swap 2 number a & b.

```
#include <stdio.h>
void swap(int a, int b);
int main() {
    int a = 3, b = 5;
    swap(a, b);
    return 0; printf("x = %d & y = %d\n", x, y);
    return 0;
}
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
    printf("a = %d & b = %d\n", a, b);
}
```

Output :- a = 5, b = 3
x = 3, y = 5

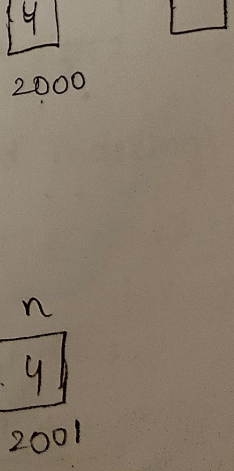


```
#include <stdio.h>
void swap(int *a, int *b);
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int main() {
    int x = 3, y = 5;
    swap(&x, &y);
    printf("x = %d & y = %d\n", x, y);
    return 0;
}
```



P-Q-41:- Will the address output be same?

```
Void printAddress(int n);
int main() {
    int n = 4;
    printf("%p\n", &n);
    printAddress(n);
    return 0;
}
Void printAddress(int n) {
    printf("%p\n", &n);
}
```



No, address is not same.

If we write the above with the help of pointers then the output will be same.

P-Q-42 :- Write a function to calculate the sum, product and average of 2 numbers. Print that average in the main function.

```
#include <stdio.h>

void doWork(int a, int b);

int main() {
    int a = 3, b = 5;
    int sum, prod, avg;
    doWork(a, b, &sum, &prod, &avg);
    printf("sum = %d, prod = %d, avg = %d\n", sum, prod, avg);
    return 0;
}
void doWork(int a, int b, int *sum, int *prod, int *avg) {
    *sum = a+b;
    *prod = a*b;
    *avg = (a+b)/2;
}
```

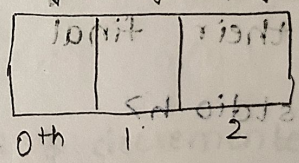
CH-7: ARRAYS

Arrays:- Collection of similar data types stored at contiguous memory locations.

```
#include <stdio.h>
int main(){
 int marks1 = 97;
 int marks2 = 98;
 int marks3 = 89;
 int marks[] = {97, 98, 89}; → stores the same data type values
 return 0;
}
```

Syntax:-

```
int marks[3];
char name[10];
float price[2];
```



(0-based indexing)

Input & output:-

```
scanf("%d", &marks[0]);
Printf("%d", marks[0]);
```

```
#include<stdio.h>
int main(){
 int marks[3];
 Printf("Enter phy:");
 scanf("%d", &marks[0]);
 Printf("Enter chem:");
 scanf("%d", &marks[1]);
```

```
Printf (" Enter math : ");
scanf ("%d", &marks[2]);
Printf ("phy = %d , chem = %d, math = %d", marks[0],
        marks[1], marks[2]);
return 0;
}
```

Output:-

Enter phy : 97
Enter chem : 98
Enter math : 99.

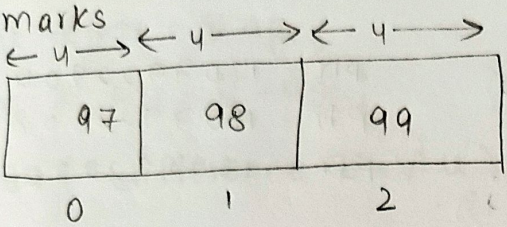
Phy = 97, chem = 98, math = 99

P-Q-43:- Write a program to enter price of 3 items and print their final cost.

```
#include <stdio.h>
int main(){
float price[3];
printf (" Enter 3 prices : ");
scanf ("%f", &price[0]);
scanf ("%f", &price[1]);
scanf ("%f", &price[2]);
printf (" Total price 1 :%f\n", price[0] + (0.18 * price[0]));
printf ("Total price 2 :%f\n", price[1] + (0.18 * price[1]));
printf ("Total price 3: %f\n", price[2] + (0.18 * price[2]));
return 0;
}
```

Intitalization of Array:-

```
int marks [ ] = {97, 98, 99};
```
(or)
```
int marks [3] = {97, 98, 99};
```



Memory reserved = 4+4+4 = 12 bytes

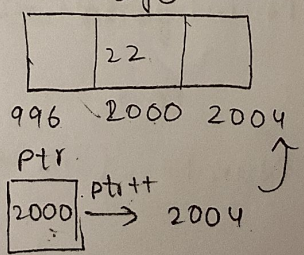
```
# include <stdio.h>
int main() {
float price [ ] = {100.0, 200.0, 300.0};
return 0;
}
```

Pointer Arithmetic :-

Pointer can be incremented & decremented.

case(i) :-

```
int age = 22;
int *ptr = &age;
ptr ++;
```



Note :-

```
#include <stdio.h>
int main() {
  int age = 22;
  int *ptr = &age;
  Printf ("ptr = %u\n", ptr);
  ptr ++;
  Printf ("ptr = %u\n", ptr);
  ptr --;
  Printf ("ptr = %u\n", ptr);
  return 0; }
```

ptr = 182696112

Case 2 :- (+4)

```
float price = 20.00;
float *ptr = &price;
ptr++;
printf("ptr = %u\n", ptr);
ptr++;
Printf("ptr = %u\n", ptr);
ptr--;
printf("ptr = %u\n", ptr);
return 0;
}
```

ptr = 1807922904
ptr = 1807922908
ptr = 1807922908

Case 3 :- (+1)

```
char star = '*';
char *ptr = &star;
printf("ptr = %u\n", ptr);
ptr++;
printf("ptr = %u\n", ptr);
Ptr--;
Printf("ptr = %u\n", ptr);
return 0;
}
```

ptr = 1827256027
ptr = 1827256028
ptr = 1827256027

Note :-

→ we can also subtract one pointer from another

→ we can also compare two pointers

→ whenever we are subtracting two pointers then those two are in same data type.

```
#include <stdio.h>
int main(){
  int age = 22;
  int _age = 23;
  int *ptr = &age;
  int *_ptr = &_age;
printf("difference = %u\n", ptr - _ptr);
_ptr = &age;
printf("comparision = %u\n", ptr == _ptr);
return 0;
}
```

output :-

```
difference = 1
comparison = 1
```

Actually it is 4 bytes but it shows the difference of integers only.

Array is a pointer:-

```
int *ptr = &arr[0];
int *ptr = arr;
```

} represents the same

→ It points to the 1st element of the array.

Traverse an array:-

```
int aadhar[10];
int *ptr = &aadhar[0];
```

```
#include <stdio.h>
int main() {
  int aadhar[5];
  //input
  int *ptr = &aadhar[0];
  for (int i=0; i<5; i++){
      printf("%d index :", i);
      scanf("%d",(ptr+i));
                (or)
                &aadhar[i];
    }
//output
for(int i=0; i<5; i++){
    Printf("%d index = %d", i, *(ptr+i));
                          or
                          aadhar[i]
    }
  return 0;
  }
```

Arrays as function Argument:-

```
// Function declaration
void printNumbers (int arr[], int n)
                  (or)
void printNumbers (int *arr, int n);

// Function Call
printNumbers (arr, n);
```

```c
#include <stdio.h>
void printNumbers(int arr[], int n);   // or *arr
int main() {
  int arr[] = {1,2,3,4,5,6};
  printNumbers(arr, 6);
  return 0;
}
void printNumbers(int arr[], int n) {   // or *arr
  for(int i=0; i<n; i++){
    printf("%d\t", arr[i]);
  }
  printf("\n");
}
```

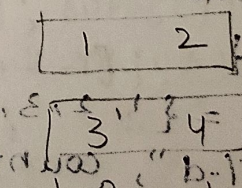
Multidimensional Arrays:-

2D arrays:-

```c
int arr[][] = {{1,2},{3,4}};
```

// Access

arr[0][0]	→	1
arr[0][1]	→	2
arr[1][0]	→	3
arr[1][1]	→	4



```
#include <stdio.h>
int main() {
int marks[2][3];
marks[0][0] = 90;
marks[0][1] = 89;
marks[0][1] = 76;

marks[1][0] = 90;
marks[1][1] = 89;
marks[1][2] = 78;

marks[2][0] =
marks[2][1] =
ma
printf("%d", marks[0][0]);
return 0;
}
```

---

p-Q-44:-

Write a function to count the number of odd no. in an array.

```
#include<stdio.h>
int main(){
  int arr[] = {1, 2, 3, 4, 5, 6};
  printf("%d", countOdd (arr, 6));
  return 0; }

int countOdd (int arr[], int n){
  int count = 0;
  for (int i = 0; i<n ; i++) {
     if (arr[i] % 2 != 0) {
        count ++;
     }

  }
return count;
}
```

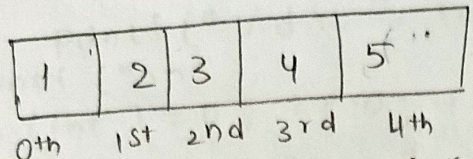
P-Q-45 :-

For the given array, what will the following give?

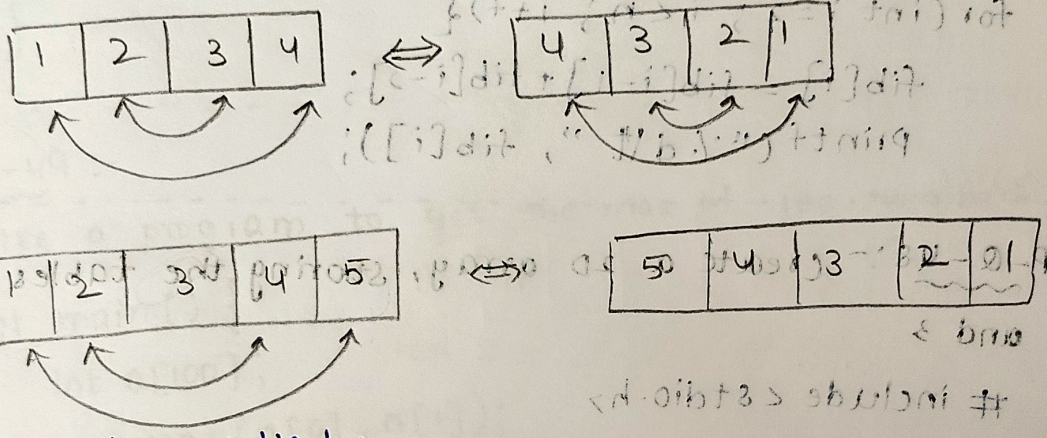
int arr[] = {1, 2, 3, 4, 5}

(a). *(arr + 2) ⟶ 3

(b). *(arr + 5) ⟶ 0



P-Q-46 :- Write a function to reverse an array.




```
#include <stdio.h>
void reverse(int arr[], int n){
    for(int i = 0; i < n/2; i++){
        int firstVal = arr[i];
        int secondVal = arr[n-i-1];
        arr[i] = secondVal;
        arr[n-i-1] = firstVal;}

int printArr(int arr[], int n){
    for(int i = 0; i < n; i++){
        printf("%d\t", arr[i]);
    }
    printf("\n");
}
int main(){
    int arr[] = {1, 2, 3, 4, 5};
    reverse(arr, 5);
    printArr(arr, 5);
    return 0;}
```

P-Q-47:- Write a program to store the first n fibonacci numbers.

```
#include <stdio.h>
int main(){
  int n;
  printf("enter n(n>2): ");
  scanf("%d", &n);

  int fib[n];
  fib[0] = 0;
  fib[1] = 1;
  for(int i=2; i<n; i++){
      fib[i] = fib[i-1] + fib[i-2];
      printf("%d\t", fib[i]);
```

P-Q-48:- create a 2D array, storing the tables of 2 and 3

```
#include <stdio.h>
void storeTable(int arr[][], int n, int m, int number
int main(){
  int Tables[2][10];
  storeTable(tables, 0, 10, 2);
  storeTable(tables, 1, 10, 3);
  return 0;
}
void storeTable(int arr[][10], int n, int m, int numb
  for(int i=0; i<m; i++){
      arr[n][m] = number*(i+1);
  }
}
```

```
int main() {
  int tables [2][10];
  storeTable (tables, 0, 10, 2);
  storeTable (tables, 0, 10, 3);
  for(int i = 0; i<10; i++) {
      printf ("%d\t", tables[0][i]);
  printf ("\n");
  for (int i = 0; i<10; i++) {
      printf ("%d\t", tables [ ][ ]);
    }
   return 0;
  }
```

P-Q-49 :-

Write a program to get the max of 100 numbers.

```
int main () {
   int a[100];
   m = max (a[0], a[1]);
   for (i = 2; i < 100; i++) {
      m = max(a[i], m);
      return 0;
      }
```

P-Q-50 :- write a program to store i+1 in a[i]

```
#include < stdio.h >
int main () {
 int i;
 int a[5];
 for (i = 0; i <5; i = i+1) {
      a[i] = i+1;
    printf ("%d", a[i]);
  }
 return 0;
  }
```

CH-8 : STRINGS

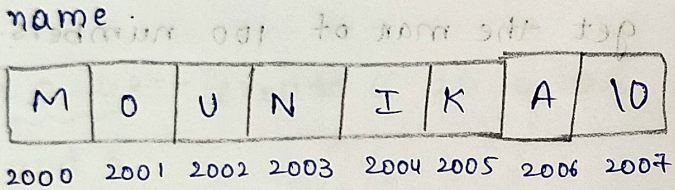
Strings :- A character array terminated by a '\0' (null character).

null character denotes string termination.

Ex :-

```
char name[] = { 'M', 'O', 'U', 'N', 'I', 'K', 'A', '\0' };
char class[] = {'C', 'O', 'L', 'L', 'E', 'G', 'E', '\0'};
char name[] = "MOUNIKA";
char class[] = "COLLEGE";
```

what happens in memory ?



P_Q-51 :- Create a string firstName & lastName to store details of user and print all the characters using a loop.

```
#include <stdio.h>
void printString(char arr[]) {
    for(int i=0; arr[i]!='\0'; i++) {
        printf("%c", arr[i]);
    }
    printf("\n");
}
int main() {
    char firstName[] = "MOUNIKA";
    char lastName[] = "KOTAPATI";
    printString(firstName);
    printstring(lastName);
    return 0;
}
```

String format specifier:- (%s)

```
char name[] = "Mounika";
printf("%s", name);
```

```
#include <stdio.h>
int main(){
    char name[50];
    scanf("%s", name);
    printf("your name is %s", name);
    return 0;
}
```

→ scanf() cannot input multi-word strings with spaces

→ Here gets() & puts() come into picture

String functions:-

gets(str) → Dangerous & out dated

input a string (even multiword)

puts(str)

output a string

```
char str[100];
gets(str);
puts(str);
```

fgets(str, n, file) → stops when n-1 chars input or new line is entered.

```
char str[100];
fgets(str, 100, stdin);
puts(str);
```

Strings using pointers :-

```
char *str = "Hello World";
```

Store string in the memory and the assigned address is stored in the char pointer 'str'.

```
char *str = "Hello world";   // can be reinitialized
char str[] = "Hello World";  // cannot be reinitialized.
```

Ex :-

```
char * canChange = "Hello World";
puts (canChange);
canChange = "Hello";
puts (canChange);
```

Output :-
Hello World
Hello

```
char cannotChange[] = "Hello World";
puts(cannotChange);
cannotChange = "Hello"
```

Output :-
red line

P-Q-52 :- Make a program that inputs user's name and prints its length.

```
# include <stdio.h>
int countLength(char arr[]);
int main() {
    char name[100];
    fgets (name, 100, stdin);
    printf ("length is %d", countLength(name));
    return 0;
}
int countLength(char arr[]){
    int count = 0;
for(int i = 0; arr[i] != '\0'; i++) {
        count++;}
    return count-1;
}
```

Standard library functions:-

<string.h>

→ ① strlen (str)
count number of characters excluding '\0'

Ex:-
```
#include <stdio.h>
#include <string.h>
int main() {
    char name[] = "MOUNIKA";
    int length = strlen(name);
    printf(" length is %d ", length);   // 7
    return 0;
}
```

② strcpy (newstr, oldstr)
(copies value of old string to new string)

Ex:-
```
#include <stdio.h>
#include <string.h>
int main() {
    char oldstr[] = "oldstr";
    char newstr[] = "newstr";
    strcpy (newstr, oldstr);
    puts (newstr);   // oldstr
```

③. strcat (firstStr , secstr)

Concatenates first string with second string.

Ex:-

```
#include <stdio.h>
#include <string.h>
int main() {
  char firstStr[100] = "Hello";
  char secStr[] = "World";
  strcat (firstStr , secStr);
  puts(firstStr);                  // combines both strings
}
```

④. strcmp(firstStr , secStr)

(compares 2 strings and returns a value)

0 → string equal

+ve → first > second (ASCII)

-ve → first < second (ASCII)

Ex:-

```
#include <stdio.h>
#include <string.h>
int main() {
  char firstStr[] = "Apple";
  char secStr[] = "Banana";
  printf ("%d", strcmp( secStr , firstStr ));
}
// +ve
```

P-Q-53:- Take a string input from the user using %c?

```
#include <stdio.h>
#include <string.h>
int main(){
    char str[100];
    char ch;
    int i=0;
    while(ch!='\n'){
        scanf("%c",&ch);
        str[i]=ch;
        i++;
    }
    str[i]='\0';
    puts(str);
}
```

P-Q-54:- write a function named slice, which takes a string and returns a sliced string from index n to m.

```
#include<stdio.h>
void slice(char str[], int n, int m){
    char newstr[100];
    int j=0;
    for(int i=n; i<=m; i++,j++){
        newstr[j]=str[i];
    }
    newstr[j]='\0';
    puts(newstr);
}
int main(){
    char str[]="HelloWorld";
    slice(str,3,6);
}
```

P-Q-55:- Write a function to count the occurence of vowels in string.

```
#include <stdio.h>
#include <string.h>
int countVowels(char str[]){
    int count = 0;
  for(int i = 0; str[i] != '\0'; i++) {
      if(str[i] == 'a' || str[i] == 'e' || str[i] == 'i' ||
          str[i] == 'o' || str[i] == 'u') {
           count ++;
       }
    }
    return count;
  }
  int main(){
    char str[] = "Helloworld";
    printf("vowels are %d", countVowels(str));
  }
```

P-Q-56:- Check wheather the character is present in the string or not.

```
#include <stdio.h>
void checkChar(char str[], char ch){
    for ( int i = 0; str[i] != '\0'; i++) {
          if (str[i] == ch) {
              printf("character is present");
              return;
          }
    }
    printf("character is not present");
}
```

```
int main(){
    char str[] = "ApnaCollege";
    char ch = 'x';
    checkChar(str, ch);
}
```

CH-9 : STRUCTURES

Structures :- A collection of values of different data types

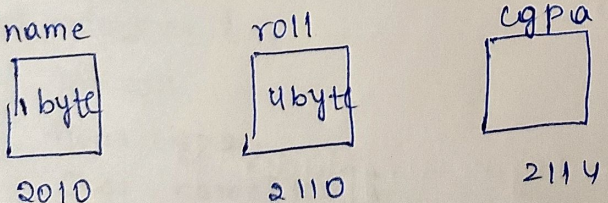
Ex :- For a student store the following:

name (string)
roll no. (integer)
cgpa (float)

Syntax :-

```
#include <stdio.h>
#include <string.h>
struct student {
    int roll;
    float cgpa;
    char name[100];
};
int main() {
struct student s1;
s1.roll = 1664;
s1.cgpa = 9.2;
strcpy (s1.name, "Mounika");
printf ("student name = %s", s1.name);
printf ("student roll no = %d", s1.roll);
printf ("student cgpa = %f", s1.cgpa);
return 0;
}
```

Structures are stored in contiguous memory location




```
Struct student {
    char name[100];
    int roll;
    float cgpa;
};
```

Array of structures :-

```
Struct student ECE[100];
Struct student CSE[100];
Struct student IT [100];
```

Access :-

```
IT[0].roll = 200;
IT[0].CGPA = 7.6;
```

```
#include <stdio.h>
#include <string.h>
Struct student{
    int roll;
    float cgpa;
    char name[100];
};
int main(){
    Struct student ece[100];
    ece[0].roll = 1664;
    ece[0].cgpa = 9.2;
    strcpy(ece[0].name, "Mounika");
    printf("name = %.s\n", ece[0].name);
    return 0;
}
```

Initializing structures :-

```
Struct student s1 = {"Mounika", 1664, 8.2};
Struct student s2 = {"Likhitha", 1234, 9.0};
Struct student s3 = {0};
```

```
#include <stdio.h>
#include <string.h>
Struct student{
    int roll;
    float cgpa;
    char name[100];
};
int main(){
    struct student s1 = {1664, 8.2, "Mounika"};
    printf("student roll = %.d\n", s1.roll};
    return 0;
}
```

Pointers to structures:-

```
struct student S1;
struct student *ptr;
ptr = &S1;

int main() {
    struct student S1 = {1664, 8.2, "Mounika"};
    struct student *ptr = &S1;
    printf("student roll = %d \n", (*ptr).roll);
    return 0;
    }
```

Arrow Operator:-

(*ptr).code ⟷ ptr->code

```
int main() {
    struct student S1 = {1664, 8.2, "Mounika"};
    printf("student -> roll = %d \n", ptr -> roll);
    return 0;
    }
```

Passing structure to function:-

```
// function prototype
void printInfo(struct student S1);
```

Ex:-

```
# include <stdio.h>
# include <string.h>
void printInfo( struct student S1
struct student {
        int roll;
        float cgpa;
        char name[100];
    };
void printInfo(struct student S1);
```

```
int main() {
    struct student s1 = { 1664, 8.2, "Mounika" };
    Printf("student roll = %d\n", s1.roll);
    PrintInfo(s1);
    return 0;
}
void printInfo(struct student s1) {
    printf("student Information :\n");
    printf("student roll no. : %d\n", s1.roll);
    printf("student name : %s\n", s1.name);
    printf("student cgpa : %f\n", s1.cgpa);
}
```

Typedef Keyword :-

Used to create alias for data types.

```
type def struct ComputerScience EngineeringStudent{
    int roll;
    float cgpa;
    char name[100];
} coe;
```

Ex:-

```
#include <stdio.h>
#include <string.h>
typedef struct student {
    int roll;
    float cgpa;
    char name[100];
} stu;

typedef struct computer Engineering student{
    int roll;
    float cgpa;
    char name[100];
} coe;
```

```
int main() {
    coe s1;
    s1.roll = 1664;
    s1.cgpa = 9.2;
    strcpy(s1.name, "Mounilca");
    printf("student name : %s\n", s1.name);
    return 0;
}
```

PQ-58 :- Enter address (house no, block, city, state) of 5 people.

```
#include <stdio.h>
struct address {
    int houseno;
    int block;
    char city[50];
    char state[100];
};
int main() {
    struct address adds[5];
    printf("enter info for person 1: ");
    scanf("%d", &adds[0].houseNo);
    scanf("%d", &adds[0].block);
    scanf("%s", adds[0].city);
    scanf("%s", adds[0].state);
    printAdd(adds[0]);
    return 0;
}
void printAdd(struct address add) {
    printf("Adress is : %d, %d, %s, %s", add.houseno, add.block,
                                        add.city, add.state);
}
```

P.Q:-59:- Create a structure to store vectors. Then make function to return sum of 2 vectors.

```
#include <stdio.h>

struct vector {
    int x;
    int y;
};
void calsum(struct vector v1, struct vector v2, struct vector sum
int main() {
    struct vector v1 = {5, 10};
    struct vector v2 = {3, 7};
    struct vector sum = {0};
}

Void calsum(struct vector V1, struct vector V2, struct vector
    Sum.x = V1.x + V2.x;
    sum.y = v1.y + V2.y;
    Printf("sum of x is %d \n", sum.x);
    Printf("sum of y is %d \n", sum.y);
}
```

P.Q-60:- Create a structure to store complex numbers. (use arrow operator).

```
struct complex {
    int real;
    int img;
};

int main(){
    struct complex n1 = {5, 8};
    struct complex *ptr = &n1;
    printf("real part = %d\n", ptr->real);
    Printf("Imaginary part = %d\n", ptr->img);
    return 0;
}
```

P-Q-61:- You have to store the marks of 30 students in class. what will you use?

(a). Array of 30 floats

(b). Structure.

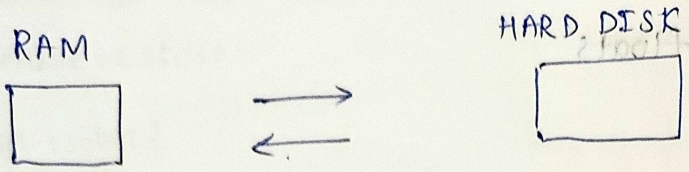
P-Q-62:- Make a structure to store bank account information of a customer of ABC bank. Also, make an alias for it.

```
#include <stdio.h>

typedef struct BankAccount {
    int accountno;
    char name[100];
} acc;

int main() {
    acc acc1 = {123, "Shradha"};
    acc acc2 = {345, "Mounika"};
    acc acc3 = {567, "Likhitha"};
printf("acc no: %d", acc1.accountno);
printf("name = %s", acc1.name);
 return 0;
}
```

CH-10: FILE - INPUT / OUTPUT



→ volatile

→ contents are lost when program terminates

→ Files are used to persist the data

Operation on files :-

(i). Create a file
(ii). Open a file
(iii). Close a file
(iv). Read from a file
(v). Write in a file

Types of files :-

Text files	Binary files
.txt, .c	.exe, .mp3, .jpg
textual data	binary data

File pointer :-

FILE is a (hidden) structure that needs to be created to opening a file.

A FILE Ptr that points to this structure & is used to access the file

```
FILE *fptr;
```

Opening a file :-

```
FILE **fptr;
fptr = fopen("filename", "mode");
```

closing a file :-

```
fclose(fptr);
```

file operating modes :-

"r"	→ open to read	} no file → null
"rb"	→ open to read in binary	
"w"	→ open to write	} no file → create. (prev data will be deleted)
"wb"	→ open to write in binary	
"a"	→ open to append.	→ (not deleted).

P-Q-63 :-

Check if a file exists before reading from it.

```
#include <stdio.h>
int main(){
   FILE *fptr;
   fptr = fopen("NewTest.txt","r");
   if(fptr == NULL){
       printf("file doesnot exist\n");
    }
   else{
    fclose(fptr);
   }
   return 0;
   }
```

Reading from a file :-

```
char ch;
fscanf(fptr, "%c", &ch);
```

Writing to a file :-

```
char ch = 'A';
fprintf(fptr, "%c", ch);
```

Read and write a char :-

```
fgetc(fptr)
fputc('A', fptr);
```

EOF (End of File) :-

fgetc returns EOF to show that the file has ended

```
#include <stdio.h>
int main() {
 FILE *fptr;
 fptr = fopen("Test.txt","r");
 char ch;
 ch = fgetc(fptr);
 while(ch != EOF) {
    printf("%c", ch);
    ch = fgetc(fptr);
 }
 printf("\n");
 fclose(fptr);
 return 0;
}
```

P-Q:-64:- Make a program to read 5 integers from a file.

```
#include <stdio.h>
int main(){
   FILE *fptr;
   fptr = fopen("test.txt", "r");
   int n;
   fscanf(fptr, "%d", n);
   printf("number = %d\n", n);
   fscanf(fptr, "%d", n);
   printf("number = %d\n", n);
   fscanf(fptr, "%d", n);
   printf("number = %d\n", n);
   fscanf(fptr, "%d", n);
   printf("number = %d\n", n);
   fclose(fptr);
   return 0;
}
```

P-Q:-65:- Make a program to input student information from a user and enter it to a file.

```
#include <stdio.h>
int main(){
   FILE *fptr;
   fptr = fopen("student.txt", "w");
   char name[100];
   int age;
   float cgpa;
   Printf("enter name: ");
   scanf("%s", &name);
   printf("enter age: ");
   scanf("%d", &age);
   printf("enter CGPA: ");
   scanf("%f", &cgpa);
   fprintf(fptr, "%s\t", name);
   fprintf(fptr, "%d\t", age);
   fprintf(fptr, "%f\t", cgpa);
   fclose(fptr);
```

P-Q:-66:- Write a program to write all the odd numbers from 1 to n in a file.

```
#include<stdio.h>
int main() {
    FILE *fptr;
    fptr = fopen("odd.txt","w");
    int n;
    printf("enter n: ");
    scanf("%d", &n);
    for(int i=1; i<=n; i++) {
        if(i%2!=0) {
            fprintf(fptr,"%d\n",i);
        }
    }
    fclose(fptr);
    return 0;
}
```

P-Q:-67:- 2-numbers - a & b are written in a file. Write a program to replace them with their sum.

```
#include<stdio.h>
int main() {
    FILE *fptr;
    fptr = fopen("sum.txt","r");
    int a;
    fscanf(fptr, "%d", &a);
    int b;
    fscanf(fptr, "%d", &b);
    fclose(fptr);
    fptr = fopen("sum.txt","w");
    fprintf(fptr, "%d", a+b);
    fclose(fptr);
    return 0;
}
```

CH-11 : DYNAMIC MEMORY ALLOCATION

It is a way to allocate memory to a data structure during the runtime.
We need some functions to allocate & free memore dynamically

Functions for DMA :-

(a). malloc() memory allocation
(b). calloc() continuous allocation
(c). tree()
(d). realloc() re-allocation

Malloc :-

Takes number of bytes to be allocated and returns a pointer of type void.

```
ptr = (int*) malloc(5 * sizeof(int))
```

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *ptr;
    ptr = (int *) malloc(5 * sizeof(int));
    ptr[0] = 1;
    ptr[1] = 2;
    ptr[2] = 3;
    ptr[3] = 4;
    ptr[4] = 5;

    for(int i=0; i<5; i++) {
        printf("%d\n", ptr[i]);
    }
    return 0;
}
```

P-Q:-68:-

WAP to allocate memory to store 5 prices.

```
#include <stdio.h>
int main() {
    float *ptr;
    ptr = (float *)malloc (5 * sizeof(float));
    ptr[0] = 1;
    ptr[1] = 2;
    ptr[2] = 3;
    ptr[3] = 4;
    ptr[4] = 5;
    for(i=0; i<5; i++) {
        printf("%f\n", ptr[i]);
    }
    return 0;
}
```

Calloc():-

Intializes with 0

```
ptr = (int *) calloc(5, sizeof(int));
```

```
#include<stdio.h>
int main() {
    float *ptr;
    ptr = (float*) malloc(5 * sizeof(float));
    for (int i=0; i<5; i++) {
        printf("%f\n", ptr[i]);
    }
    return 0;
```

Output :-

```
0.000
0.000
0.000
0.000
0.000
```

P-Q :- 69 :- WAP to allocate memory of size n., where n is entered by the user

```
int *ptr;
 int n;
printf("enter n : ");
scanf("%d", &n);
ptr = (int *) calloc (n, sizeof (int));
for(int i=0; i<n ; i++) {
   printf("%d\n", ptr[i]);

 }
return 0;
}
```

free() :-

We use it to free memory that is allocated using malloc & calloc.

```
free(ptr);
```

P.Q :- 70 :-

Free the memory allocated with calloc

```
#include < stdio.h>
int main(){
 float *ptr;
 ptr = (float *)calloc (5 , sizeof (float));
 for(int i=0 ; i<n ;  i++){
      printf("%d\n", ptr[i]);
 }
 free(ptr);
  }
```

realloc() :-

reallocate (increase or decrease) memory using the same pointer and size.

```
ptr = realloc(ptr, newsize);
```

P-Q :- 71:- Allocate memory for 5 numbers. Then ~~individual~~ dynamically inc it to 8 numbers

```
#include <stdio.h>
#include <stdlib.h>
int main(){
  int *ptr;
   ptr = (int *)calloc(5, sizeof(int));
  printf("enter no.s (5): ");
  for(int i=0; i<5; i++){
      scanf("%d", &ptr[i]);
  }
 ptr = realloc(ptr, 8);
 printf("enter no.s (8): ");
 for(int i=0; i<8; i++){
      scanf("%d", &ptr[i]);
   }
 for(int i=0; i<8; i++){
      printf("number %d is %d", i, ptr[i]);
   }
 return 0;
  }
```

P-Q:-72:- Create an array of size 5 (using calloc) & enter its values from the user.

```
int *ptr;
ptr = (int *)calloc(5, sizeof(int));
printf("enter nos ");
for(int i=0; i<5; i++){
    scanf("%d", &ptr[i]);
}
```

P-Q:-73 :- Allocate memory to store 1st 5 odd numbers, then reallocate it to store 1st 6 even nos.

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int *ptr;
    ptr = (int *)calloc(5, sizeof(int));
    ptr[0] = 1;
    ptr[1] = 3;
    ptr[2] = 5;
    ptr[3] = 7;
    ptr[4] = 9;
    for(int i=0; i<5; i++){
        printf("%d\n", ptr[i]);
    }
ptr = realloc(ptr, 6);
ptr[0] = 2;
ptr[1] = 4;
ptr[2] = 6;
ptr[3] = 8;
ptr[4] = 10;
ptr[5] = 12;
for(int i=0; i<6; i++){
    printf("%d\n", ptr[i]);
}
return 0;
}
```